TRANSPORT AND TELECOMMUNICATION INSTITUTE

ENGINEERING FACULTY

Control test Nr. 1

Individual Variant Nr. 1

Course: **Programming Languages Concepts**

Theme: **PL Syntax**

Student: Igors Oļeiņikovs

Student code: 93642

Group: 4501BTA

Riga
2026

CONTENTS

1. TASK 1 3
2. TASK 2 4
3. TASK 3 5
4. TASK 4 6
5. TASK 5 8

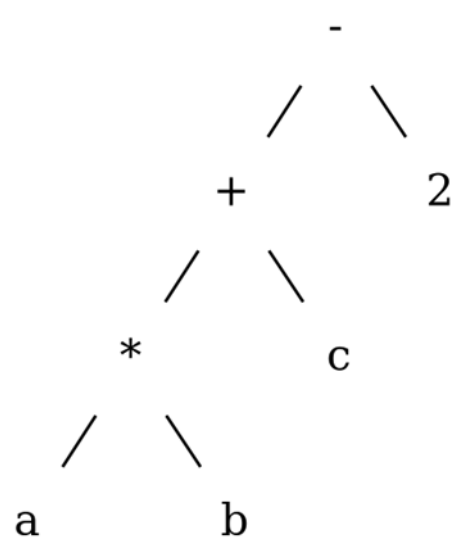
1. TASK 1

Represent expressions in prefix and postfix notations and draw an abstract syntax tree.

<u>Expression 1</u>

a * b + c - 2

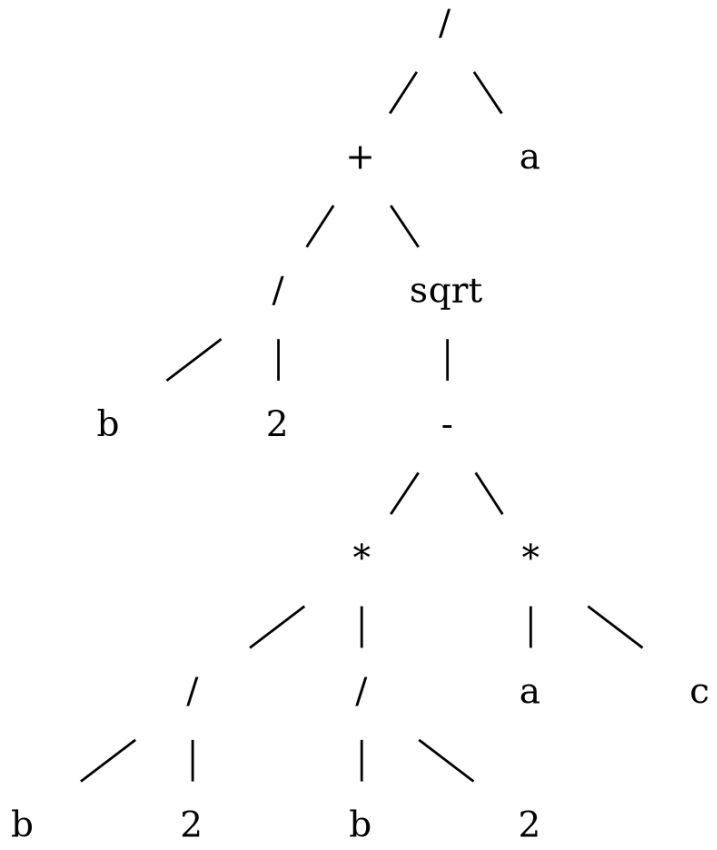
- Prefix notation: **- + * a b c 2**
- Postfix notation: **a b * c + 2 -**
- **<u>Abstract syntax tree:</u>**



<u>Expression 2</u>

(b / 2 + sqrt ((b / 2) * (b / 2) - a * c)) / a

- Prefix notation: **/ + / b 2 sqrt - * / b 2 / b 2 * a c a**
- Postfix notation: **b 2 / b 2 / b 2 / * a c * - sqrt + a /**
- **<u>Abstract syntax tree:</u>**



2. TASK 2

Get context-free grammar describing the syntax of the following construct:

Letters and/or digits sequence: a sequence has 0 length or more characters, starts with digit.

Note: the task says “0 length or more”, but an empty string cannot start with a digit, so the grammar below generates strings of length 1 or more.

```
<S> ::= <digit> <tail>
<tail> ::= <symbol> <tail> | epsilon
<symbol> ::= <letter> | <digit>
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
<letter> ::= A | B | C | ... | Z | a | b | c | ... | z
```

Note: The ellipses in <letter> represent all 52 lowercase and uppercase letters.

3. TASK 3

Using the grammar, show a parse tree and a leftmost derivation for string 1.23456.

```
<real_number> ::= <integer_part> . <fractional>
<integer_part> ::= <digit> | <integer_part> <digit>
<fractional> ::= <digit> | <digit> <fractional>
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

<u>Leftmost derivation:</u>

<real_number>

=> <integer_part> . <fractional>

=> <digit> . <fractional>

=> 1 . <fractional>

=> 1 . <digit> <fractional>

=> 1.2 <fractional>

=> 1.2 <digit> <fractional>

=> 1.23 <fractional>

=> 1.23 <digit> <fractional>

=> 1.234 <fractional>

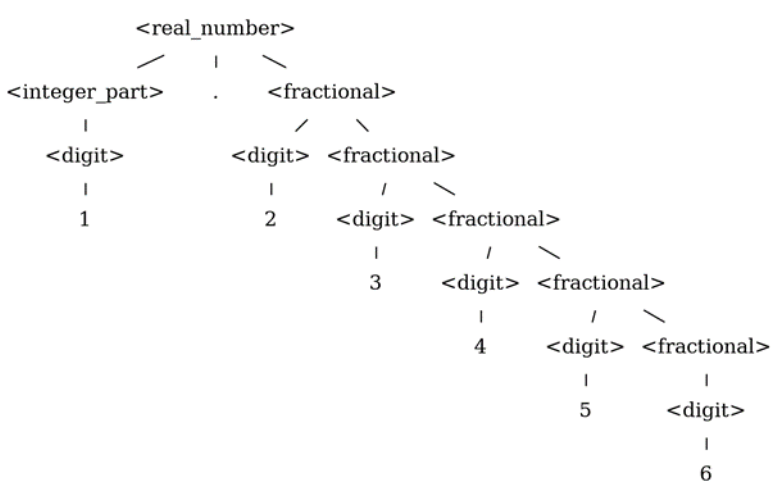
=> 1.234 <digit> <fractional>

=> 1.2345 <fractional>

=> 1.2345 <digit>

=> 1.23456

<u>Parse tree:</u>



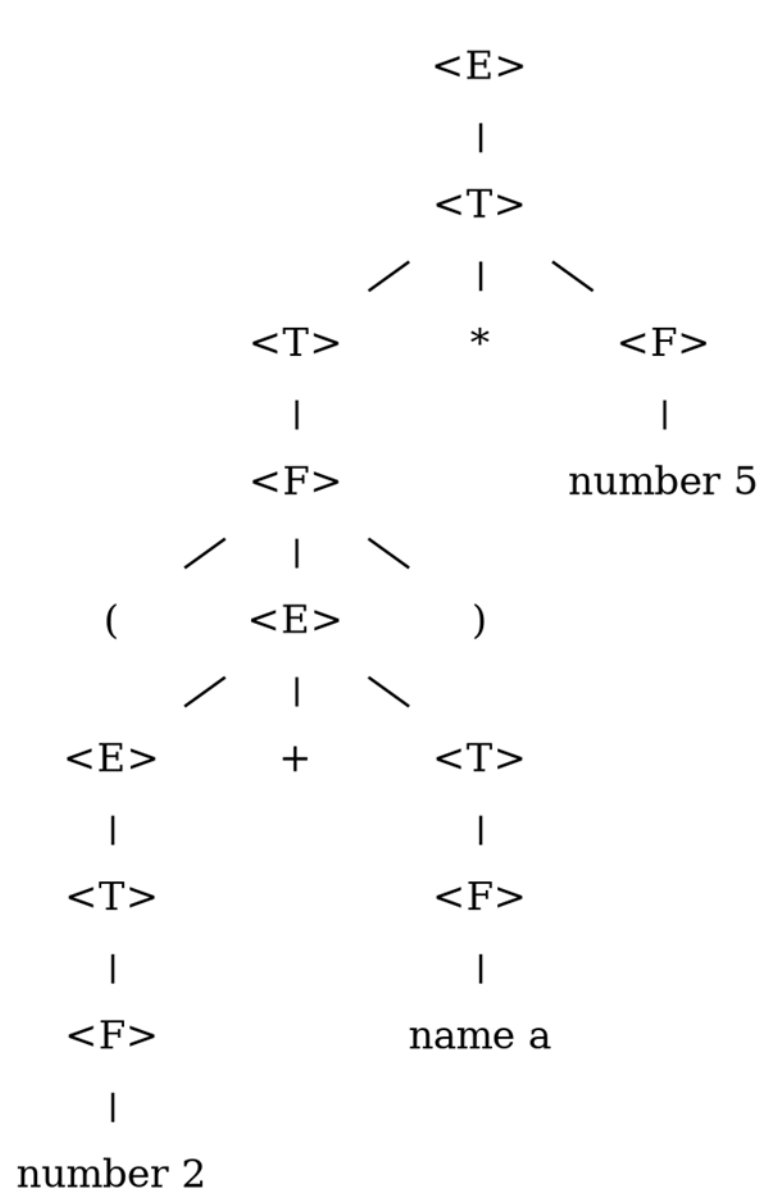
4. TASK 4

Using the grammar, show a parse tree for the given expressions.

```
<E> ::= <E> + <T> | <E> - <T> | <T>
<T> ::= <T> * <F> | <T> / <F> | <F>
<F> ::= number | name | ( <E> )
```

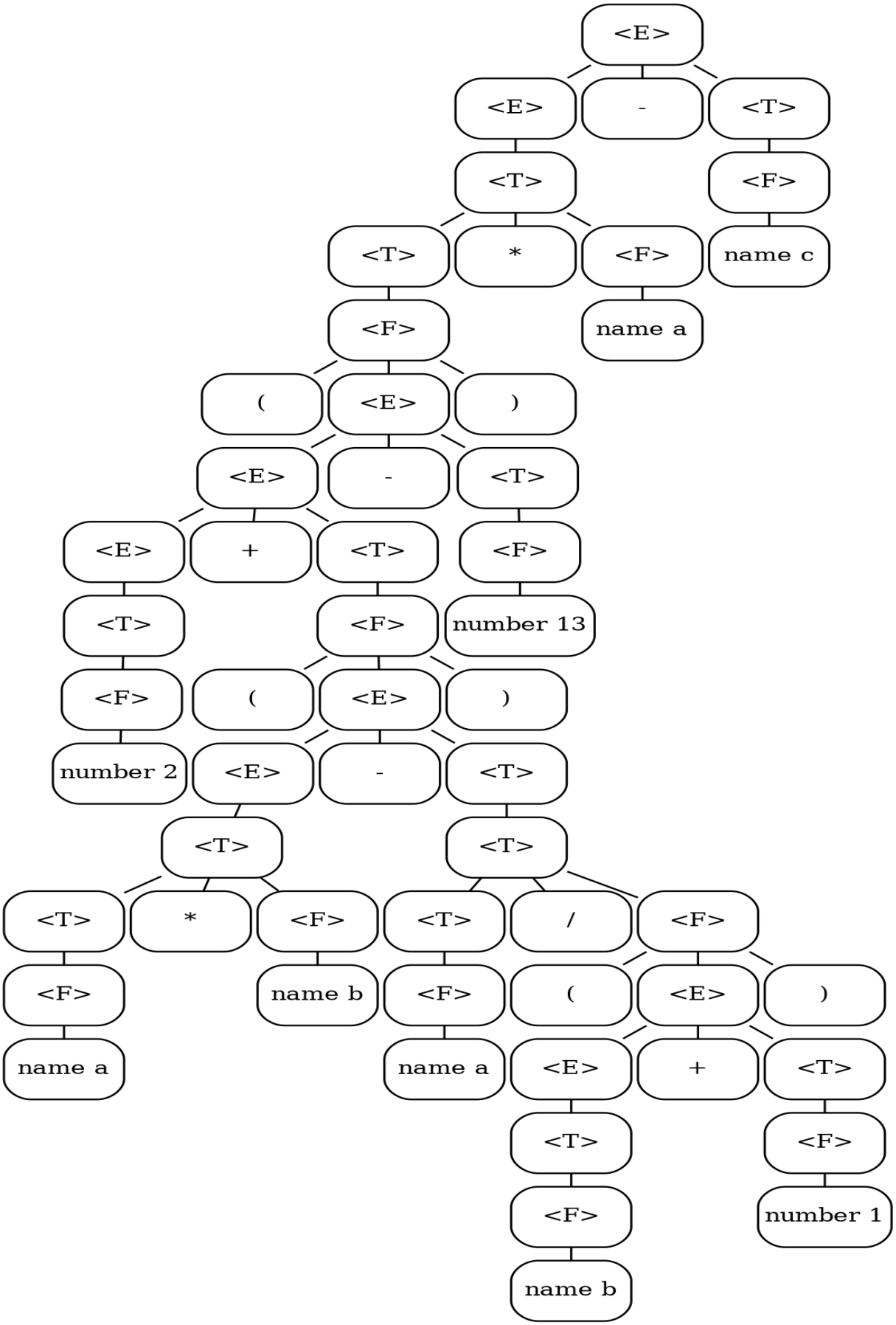
<u>Expression 1</u>

(2 + a) * 5



<u>Expression 2</u>

(2 + (a * b - a / (b + 1)) - 13) * a – c

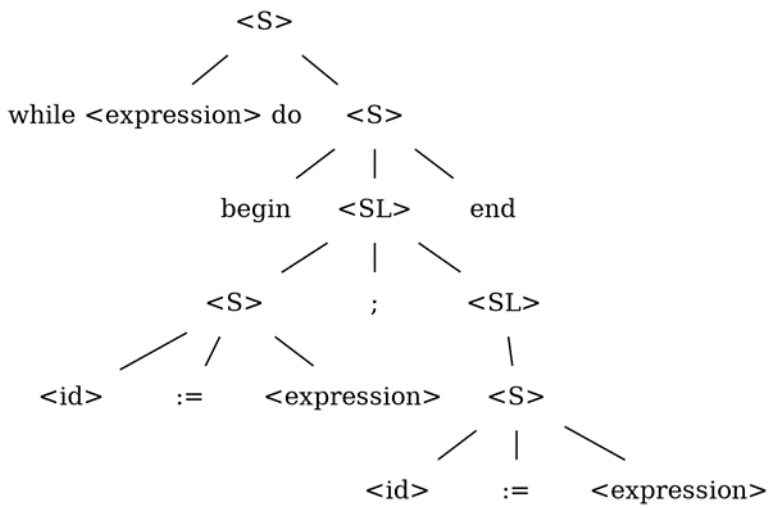

<E>
<E>
-
<T>
<T>
<F>
<T>
*
<F>
name c
<F>
name a
(
<E>
)
<E>
-
<T>
<E>
+
<T>
<F>
<T>
<F>
number 13
<F>
(
<E>
)
number 2
<E>
-
<T>
<T>
<T>
<T>
*
<F>
<T>
/
<F>
<F>
name b
<F>
(
<E>
)
name a
name a
<E>
+
<T>
<T>
<F>
<F>
number 1
name b

5. TASK 5

Using the grammar for Pascal language, show a parse tree for the given construction.

```
<S> ::= <id> := <expression>
      | if <expression> then <S>
      | if <expression> then <S> else <S>
      | while <expression> do <S>
      | begin <SL> end
<SL> ::= <S> | <S> ; <SL>
```

<u>Construction:</u>

while <expression> **do begin** <id> := <expression>; <id> := <expression> **end**

<u>Parse tree:</u>